

Research Progress Report — Router Re-scoping, Literature Review, and PROBE-CASCADE Design

Multi-LLM Comparison Platform: From Model Comparison to a Cost-Aware Router That Generalises to New Models

Student: Venu Dattathreya Vemuru (811776500) **Course:** CSCI 7200 — Master's Project **Report period:** August–September 2026 (since Research Progress Report 6) **GitHub:** https://github.com/venu284/LLM_Comparison.git

1. Executive Summary

Report 6 closed Phase 6 with a negative result. On our five-model pool, routing between models has an oracle ceiling of only +8.6 percentage points over always using the best single model (GPT-OSS-120B, 53.8%). Our category-weighted recommender scored 36.9% under leave-one-task-out validation. This report covers the work done since then to turn that result into a sound router design.

The period produced five outcomes:

1. Data sufficiency and failure analysis (early August).

- The Phase 5 data is sufficient to build on.
- The 63 zero-test rows now have a derived failure taxonomy: 10 defective-task, 10 truncated, and 43 undiagnosable.
- The top-2 router analysis shows that a *static* pair of models already reaches the oracle.

2. Direction reset (16 August).

- The project moved from *selection* routing ("which model is best?") to *sufficiency* routing ("is a cheap model good enough?").
- Ten design decisions were recorded (D1–D10), and exploratory cost analyses were run on the Phase 5 data.

3. Adversarial review of the new direction (18 August).

- The review attacked the plan's novelty, metric, and data assumptions, and computed corrections on a public 70,678-row code-routing corpus.
- Two parts of the plan were retired and one was strengthened.

4. Literature review (September).

- A survey of roughly 30 routing papers, mostly from 2025–2026, plus industry routers.

- It includes a close reading of "LLM Routing with Dueling Feedback" and "Agent-as-a-Router" (ACRouter, June 2026).

5. Proposed method: PROBE-CASCADE (September).

- A router for code generation that onboards a *new* model with a small measured probe.
- It uses the cheapest model predicted to succeed, escalates when run-time checks fail, and keeps learning from checked outcomes.
- Evaluation will use public priced benchmarks (CodeRouterBench, LLMRouterBench), with our 65 web-development tasks as the showcase domain.

No new model API calls were made in this period. Every number below comes from the stored Phase 5 data, the public corpora named, or the cited papers.

2. Work Completed

2.1 Data Sufficiency and Failure Taxonomy (1–2 August)

- **Sufficiency assessment** (`Reports/Phase5_Data_Sufficiency_Assessment.md`): the 325 Phase 5 runs are adequate for Phases 6–7 analysis. Re-collecting with stronger models would *not* help routing, for the reason given in §2.3.
- **Zero-test rows**: 63 of 325 rows (19%) have `tests_total = 0`, all scored as failures.
 - A derived taxonomy (`pipeline/analysis/failure_taxonomy.py`) reconstructs failure causes from token counts, extraction status, and test totals: 10 defective-task, 10 truncated at the 4,096-token limit, and 43 undiagnosable.
 - Dropping only the 20 artifactual rows keeps the RQ1 result significant: χ^2 $p = 2.6e-4$, Cramér's $V = 0.265$.
 - The earlier $p = 0.37$ collapse was an artefact of removing 40 of Qwen3-32B's 65 attempts.
 - The loader now supports `zero_test_treatment = keep | drop_all | drop_artifactual`.
- **Top-2 router analysis**: the planned app returned two models' answers per prompt.
 - A static pair (GPT-OSS-120B + Llama-4-Scout) reaches the 58.5% oracle exactly.
 - The deployed top-2 recommender reached 46.2%, 12.3 points *worse* than hard-coding the pair.
 - The cause is the Phase 2 pre-registered 30% latency/token weight, which ranks the most accurate model fourth of five.

2.2 Direction Reset: From Selection to Sufficiency (16 August)

The original Phase 8 "which model should I use" demo was retired, because it had nothing to demonstrate at a +8.6% ceiling. The project was re-chartered as a constraint-aware router for code generation.

Design decisions:

#	Decision
D1	Sufficiency, not selection: predict whether a cheaper model is good enough
D2	Learn from task features and a model descriptor, never from a model's identity
D3	Model descriptors are measured by probing, not written by an LLM (reopened, see §7)
D4	Black-box access only (no log-probabilities)
D5	Every learning signal is automated; no hypothesis depends on recruiting human raters
D6	The user sets the cost / latency / quality weights
D7	Cost is modelled from published prices, with a mandatory price-sensitivity sweep
D8	Nesting of the model pool is treated as a feature, not a defect
D9	Only probing and the sufficiency predictor are research; everything else stays thin
D10	Disclose data defects, never repair stored runs

Exploratory analyses on the Phase 5 data (scripts in `pipeline/analysis/exploratory/`, branch `worktree-phase8-goal-lock`):

Analysis	Result
Fixed cascade Llama-3.1-8B → GPT-OSS-120B	56.9% pass at −10.8% cost vs always GPT-OSS-120B; beats best-single on both axes (+21% latency)
Oracle cost decomposition	Raw oracle saving is −78.8%, but 68.4% of it comes from clairvoyant abstention on the 27 tasks no model solves; realistic headroom was estimated at about −25%
Cheap-model sufficiency by difficulty	80% on easy, 30% on medium, 8% on hard tasks
Probe-size curve	Descriptor accuracy rises from 63.8% ($k = 3$) to 74.4% ($k = 40$), but argmax routing on the descriptor stays near 52%
Partial credit	128 of 325 runs (39%) pass some but not all tests

Key insight:

- Our models' successes are *nested*: GPT-OSS-120B solves 35 of the 38 tasks any model solves.

- Nesting makes selection routing useless, but it *feeds* cascades. The cheap model handles easy tasks, and the strong model only has to catch the rest.

2.3 Adversarial Review of the Router Goal (18 August)

Before building anything, the new direction was stress-tested with an adversarial review. The review set out kill criteria before searching and computed results on the public LLMRouterBench corpus (70,678 per-instance code rows). Its verdicts:

Claim	Verdict	Consequence
Novelty: "measured model descriptors for unseen models"	Prior art. UniRoute, EmbedLLM, IRT-Router and Router-R1 already do this	Contribution reframed from mechanism to economics: <i>how much probing is worth paying for</i>
Metric: cost per accepted task (CPAT)	Degenerate. Unconstrained CPAT picks a 16.8%-accuracy model on LiveCodeBench	CPAT is now used only with an acceptance floor
Model pools are nested	Holds at frontier scale. Measured oracles sit 8.8–25.9 points below the independence null	The Phase 6 finding generalises
Realistic cost headroom of –25%	Understated. Sufficiency routing yields –33% to –50% CPAT on SWE-bench Verified and LiveCodeBench	Strengthens D1
15-cell pass-rate descriptor from a 12–15 task probe	Noise. About one sample per cell	Replace with an IRT ability estimate over selected anchor items
LLMRouterBench as the primary corpus	Weakened. About 41% of small models have zero cost, the matrix is ragged, and scores are binary only	Prefer fully priced subsets such as SWE-bench Verified
Probe transfer within code	A grader effect. HumanEval vs LiveCodeBench rank correlation is $\rho = -0.33$, but $+0.36$ once chain-of-thought models are excluded	A probe measures the grader as well as the model; the probe harness must match deployment

2.4 Literature Review (September)

We surveyed the 2025–2026 routing literature. It is summarised in §3 and cited in §10. Three sources were read in depth:

- **Dueling feedback.** "LLM Routing with Dueling Feedback" [1] was assessed as a candidate method.
- **Industry practice.** The Braintrust survey of production routers [27] was reviewed.
- **ACRouter.** "Agent-as-a-Router" [22] was read in full, including its appendix. Its released benchmark, CodeRouterBench (~10K coding tasks $\times$ 8 frontier models, with per-task scores and

costs, MIT licence), was confirmed to be usable as our main evaluation corpus.

2.5 Method Design (September)

The survey, the review, and our own data were combined into the PROBE-CASCADE design (§5) and an evaluation plan (§6).

3. Literature Review

3.1 What Current Routing Research Is Trying to Solve

Every router trades off the same three quantities: cost, latency, and output quality. Recent work differs in *how the router learns what it needs to know*. The main research threads:

1. Per-query predictability.

- Routers learn coarse patterns but miss per-query structure.
- "The Routing Plateau" [16] finds 21 methods on 5 benchmarks converging to a narrow band well below the oracle.
- DARS [18] shows single-sample labels are noisy estimates of capability.
- IRT-Router [9] models query difficulty and model ability explicitly.

2. Unseen models.

- UniRoute [11] represents a model by its errors on a small prompt set.
- LLM Bandit [13] uses model identity vectors.
- Router-R1 [10] uses text descriptors.
- RouteProfile [19] names the trade-off: LLM-written descriptions are coarse, while measured profiles are costly to build.

3. Learning from cheap feedback.

- Single-score bandits: PILOT [12] and BaRP [14].
- Pairwise preferences: dueling feedback [1].
- User retries as dissatisfaction: CQB-MNL [15].
- Nearly all of these simulate the feedback from benchmark labels.

4. Test-time control of the trade-off. CARROT [4], BaRP [14] and Arch-Router [23] let users move the cost/quality dial without retraining.

5. Cascades. Dekoninck et al. [20] prove threshold cascades optimal and unify routing and cascading. FrugalGPT [30] and AutoMix [31] are earlier cascade designs.

6. Routers as agents.

- Multi-round reasoning routers: Router-R1 [10].
- Answer aggregation across models: Avengers-Pro [25].
- Routing inside coding agents: ACRouter [22].

7. Evaluation honesty.

- A tuned kNN router matches complex ones [6].
- Many routers fail to beat simple baselines on LLMRouterBench [7].
- True per-query headroom can be smaller than run-to-run noise [17].

3.2 Method Families Organised by Model Representation

Our key requirement is that the router keeps working when a new model is added. What separates the families on that requirement is how each one represents a model.

Family	Model represented as	New model added	Cost handling	Examples
Fixed-identity classifiers	An output class	Retrain	Partial	RouteLLM [8], RouterDC [28], MixLLM [29], ACRouter's orchestrator [22]
Text descriptors	A written description	Immediate, but encodes reputation	Via price in the text	GraphRouter [26], Router-R1 [10]
Measured profiles	Scores on a probe set, or an ability estimate	After k probe tasks	Yes, if priced	UniRoute [11], EmbedLLM [21], IRT-Router [9], tinyBenchmarks [24]
Online memory / bandits	Accumulated outcomes	Learns over time, but never tried without exploration	Via the reward	PILOT [12], BaRP [14], LinUCB (in [22]), dueling [1]
Cascades	Price order plus a verifier	Trivial	The core lever	Dekoninck et al. [20], FrugalGPT [30], AutoMix [31]
LLM-as-router	Model names in a prompt	Via description	Weak	ACRouter's ablations [22] show router size does not matter

Each family covers part of our problem, and none covers all of it. PROBE-CASCADE (§5) combines three of them.

3.3 LLM Routing with Dueling Feedback [1]

Chiang, Ishida and Sugiyama cast routing as a contextual dueling bandit. Each round, two models answer, and only a preference between the two answers is observed.

- **Algorithm:** Feel-Good Thompson Sampling for Contextual Dueling Bandits (FGTS.CDB), with regret guarantees.
- **Model embeddings:** Category-Calibrated Fine-Tuning (CCFT) of a text encoder.
- **Evaluation:** RouterBench and MixInstruct, with preferences *simulated* by a Bradley-Terry model.

Our assessment for this project:

1. **The signal is weaker than what we have.** Code has executable tests, which give an absolute pass/fail signal. Pairwise preference discards information that a single-score bandit would use.
2. **Every duel costs two model calls**, which works against a cost objective.
3. **The preferences were simulated**, and no code release was found.
4. **It does not natively handle new models.**

Dueling feedback is the right tool for unverifiable, open-ended chat with a two-answer interface. It is not the right tool for verifiable code routing. We note that such an interface would also reintroduce the dependence on human raters that Phase 7 could not satisfy (zero votes collected).

3.4 Agent-as-a-Router / ACRouter [22]

ACRouter, from Zhou et al. (NUS, DAMO, Berkeley and others, June 2026), frames routing as a Context $\rightarrow$ Action $\rightarrow$ Feedback loop over a task stream. It has three modules:

- **Orchestrator:** a LoRA-tuned Qwen3.5-0.8B, a per-dimension best-model table, and the top-10 kNN neighbours from memory, combined by weighted vote.
- **Verifier:** an AST check, sandbox execution, tests embedded in the prompt, self-consistency, and an LLM judge.
- **Memory:** an embedding-keyed store of past outcomes.

Its benchmark, CodeRouterBench, contains 10,111 tasks across 10 coding dimensions $\times$ 8 frontier models, with per-task scores and costs.

Router (in-distribution, n = 2,919)	AvgPerf %	Perf / \$
Oracle	57.00	8.20
ACRouter	49.98	3.79
DimensionBest (static rule)	47.50	3.69
kNN retrieval	47.18	6.07
LinUCB bandit	46.84	4.38

Router (in-distribution, n = 2,919)	AvgPerf %	Perf / \$
Always Claude Opus 4.6	43.83	1.29

On the out-of-distribution agentic set (n = 176), ACRouter scores 62.50 against 57.14 for always using Opus.

Strengths:

- It is the first code-specific routing benchmark with priced, per-task outcomes.
- Its central finding is that the routing bottleneck is *missing information, not reasoning*: adding performance statistics to an LLM router gives +15.3% relative.
- Code and data are released.

Limitations relevant to our goal:

1. **The agent is not what helps.** Router size from 0.8B to 27B changes AvgPerf by only about 0.5 points, and all eight LLM routers score below the static DimensionBest rule. The gain comes from memory and verification, about 2.8 points over plain kNN.
2. **Cost is a tie-breaker.** The reward is score $- 0.1 \times \text{USD}$, so plain kNN has better performance per dollar (6.07 vs 3.79).
3. **The model pool is fixed.** The orchestrator and the rule table are keyed to eight model names, and memory only records models that were chosen. A newly added model has no data and no exploration mechanism, so it would rarely be selected. Adding models is not evaluated.
4. **There is no escalation.** A verifier failure is recorded but never triggers a retry on a stronger model.
5. **The out-of-distribution evidence is thin:** 176 tasks and a single seed. A standalone GPT-5.4 run scores 75.00 on the same split (their Appendix D.7).

Limitations 2–4 are the gaps PROBE-CASCADE targets.

3.5 Industry Routers [27]

Production gateways such as OpenRouter, Portkey, LiteLLM and Vercel AI Gateway mostly route with static rules, price or latency ordering, and fallbacks. Braintrust's position is that routing quality depends on *measured evidence from the application's own traffic*. In practice this means evaluating a new model before routing to it. Industry therefore treats a new model as a measurement problem, which supports our measured-probe design.

3.6 Consensus and Criticisms

- Simple baselines (kNN, static task-type rules, fixed cascades) are hard to beat [6, 7, 17]. Any new router must report them.
 - With nested pools, selection gains are small, which our Phase 6 result independently confirms. The remaining lever is cost through sufficiency and cascading [20].
 - Single-run labels overstate the oracle [17, 18]. Our Phase 5 data has `run_number = 1` throughout, so this caveat applies to it.
-

4. Refined Problem Statement

Build a router for code-generation tasks that:

1. **Operates where answers can be checked.**
 - Code, with web development as the showcase domain.
 - Tests give an automatic quality signal, consistent with D5.
 2. **Matches the best single model's pass rate at clearly lower cost.**
 - Cost per accepted task, subject to an acceptance floor.
 - The floor defaults to the best single model's pass rate, or can be set by the user.
 3. **Keeps working when a new model is added, without retraining.**
 - The user adds a model, for example through their own API key.
 - The router estimates the model's strengths from a small, bounded probe and routes to it sensibly from then on.
-

5. Proposed Method: PROBE-CASCADE

PROBE-CASCADE (Probe → Predict → Explore → Escalate) combines measured model profiles, online memory, exploration, and a verified cascade. It has no LLM in the routing decision.

Layer 0 — Prior.

- On registration, a model has only its price and, optionally, a text description.
- These give a weak initial estimate.

Layer 1 — Onboarding probe.

- The new model is run on k anchor tasks, selected with item response theory (IRT) for maximum information, following tinyBenchmarks [24].
- An ability estimate θ_m with uncertainty is fitted on the shared IRT scale. The k outcomes are written to memory.
- This replaces the noisy 15-cell descriptor with a one- or two-parameter estimate (§2.3).

Layer 2 — Per-task pass prediction.

- A task's difficulty b_q is predicted from its embedding.
- $P(\text{model } m \text{ passes task } q) = \sigma(\theta_m - b_q)$, the IRT form used by IRT-Router [9].
- This is corrected by the model's observed outcomes on the most similar past tasks in memory, following ACRouter's memory [22] but used as a residual correction rather than a vote.

Layer 3 — Decision with exploration.

- Thompson sampling draws a pass probability for each model from its posterior, so new or uncertain models are tried in proportion to their plausibility.
- The router chooses the **cheapest model whose sampled pass probability is at least τ** .
- τ is the user's quality/cost dial.

Layer 4 — Verify and escalate.

- The answer is checked with run-time signals only: prompt-embedded tests, sandbox execution, and an AST check.
- On failure, the task escalates to the next model in the cascade until it passes or the budget is spent. Threshold cascades of this form are optimal under the analysis in [20].

Layer 5 — Online update.

- Every verified outcome updates θ_m and adds a memory entry.
- The router improves with use, with no human feedback required.

Evidence behind each design choice:

Choice	Evidence
Measured profile over text description	UniRoute [11]; the review's HumanEval-vs-LiveCodeBench reversals (e.g. Fin-R1 at 77.4% vs 6.8%) show that reputation misleads
Difficulty $\times$ ability over category tables	Our Phase 6 data: difficulty's association with success is stronger than model choice's (Cramér's V 0.515 vs 0.288); few IRT anchor items suffice [24]
Memory correction	ACRouter's information-deficit finding [22]

Choice	Evidence
Exploration	Without it, a new model never accumulates evidence (ACRouter limitation 3)
Verified cascade	Optimality [20]; −33% to −50% CPAT sufficiency prize on priced code pools (§2.3); our fixed cascade already beats best-single (§2.2)
No LLM orchestrator	Router size does not matter, and LLM routers trail a static rule [22]

Novelty, stated honestly:

- Every component is published: IRT routing, probe profiles, memory, Thompson sampling, cascades.
- Our contribution is:
 1. combining them into a router that onboards new models with a bounded probe and then learns online inside a verified cascade;
 2. the first measurement, to our knowledge, of **how many probe tasks a new model needs** (cold-start regret as a function of k , against text-description and no-probe baselines);
 3. a cost-at-matched-accuracy comparison against ACRouter.
- A final prior-art check is scheduled before this is claimed as new (§8).

6. Evaluation Plan

Datasets.

- *CodeRouterBench* [22]: 8 frontier models. The primary evidence uses only the six dimensions scored by execution; three dimensions are scored by an LLM judge and will be reported separately.
- *LLMRouterBench* [7]: a larger pool, with zero-cost models excluded.
- *Our 65 web-development tasks*: a new out-of-distribution dimension with real tests and partial credit.

Experiments.

ID	Question	Design
E1	How cheaply can a new model be onboarded? (core)	Leave-one-model-out: hold a model out, add it mid-stream with $k \in \{0, 5, 10, 25, 50\}$ probe tasks or a text description only; measure cumulative regret and recovery time
E2	Does it cut cost at matched quality?	Cost at matched pass rate vs ACRouter, kNN, DimensionBest, LinUCB, fixed cascade, and always-best; multiple seeds with confidence intervals

ID	Question	Design
E3	How sensitive is it to verifier error?	Simulate verifier error rates of 0, 10, 20 and 30%
E4	Does it work on web development?	Run the model pool on our 65 tasks; live demo adding an unseen model through the user's own API key

Methodological safeguard. The benchmark's ground-truth score must never drive escalation during replay. That would give the cascade clairvoyant knowledge, the same inflation our oracle decomposition found for abstention (§2.2). Escalation decisions use only run-time verifier signals, and E3 quantifies how much verifier noise costs.

7. Challenges and Design Decisions

- **D3 reopened.** The probe-measured descriptor survives, but as an IRT ability estimate over selected anchors, not a 15-cell pass-rate table. The claimed contribution moves from mechanism to probe economics.
- **Metric corrected.** CPAT is always reported with an acceptance floor.
- **Dueling feedback not adopted** (§3.3). Pairwise signals are weaker than executable tests and double the per-query cost.
- **Evaluation corpus changed.** Our 5 models $\times$ 65 tasks cannot evaluate a new-model router. Evaluation moves to public priced corpora, and our benchmark becomes the web-development showcase.
- **Human evaluation deprioritised.** Phase 7 infrastructure remains available, but under D5 no hypothesis depends on it.
- **Open engineering items:**
 - Docker is unavailable locally, which blocks confirming the two suspected defective tasks (API-013, CSS-012).
 - The data-sufficiency branch is not yet merged.
 - `pytest` is not installed in the project environment.

8. Current Status and Next Steps

Status: the design is complete. Implementation of PROBE-CASCADE has not started, and no PROBE-CASCADE results exist yet.

Immediate next steps (October):

1. Final prior-art check: FlyRoute, MonoRouter, "Routing with Generated Data", and papers citing ACRouter.
2. Download CodeRouterBench and reproduce the ACRouter, kNN and DimensionBest baselines.
3. Implement the IRT fit, anchor selection, and the onboarding probe; run E1.

Timeline:

Month	Work
October 2026	Baseline reproduction; onboarding probe; E1
November 2026	Cascade and exploration layers; E2, E3; run the model pool on the 65 web-dev tasks
December 2026	E4 live demo; thesis write-up and defence

9. Threats to Validity

- **Small pool for leave-one-model-out.** CodeRouterBench has 8 models, far fewer than UniRoute's 30+. LLMRouterBench is added to widen the pool.
- **Single-sample labels.** Public corpora and our Phase 5 data record one run per model–task pair, so oracle gaps may be inflated by noise [17, 18].
- **Modelled prices.** Costs come from published token prices. Provider caching and discounts are not observed.
- **LLM-judged dimensions** in CodeRouterBench are noisy and are excluded from primary evidence.
- **Probe–grader mismatch.** A probe measures the grading harness as well as the model (§2.3). The onboarding harness must match deployment.
- **Novelty risk.** Several 2026 papers not yet read may overlap (§8, step 1).

10. References

Identifiers were confirmed against arXiv during this review unless marked (*unverified*). Titles for [3], [5], [6], [13]–[17], [19], [23] and [25] are abbreviated from the arXiv listings and will be checked against the final versions before the thesis.

[1] C.-K. Chiang, T. Ishida, M. Sugiyama, "LLM Routing with Dueling Feedback," arXiv:2510.00841, 2025. [2] Q. J. Hu et al., "RouterBench: A Benchmark for Multi-LLM Routing System," arXiv:2403.12031, 2024. [3] Z. Huang et al., "RouterEval: A Comprehensive Benchmark for Routing LLMs," arXiv:2503.10657, 2025. [4] S. Somerstep, F. Maia Polo et al., "CARROT: A Cost Aware Rate Optimal Router" (SPROUT dataset), arXiv:2502.03261, 2025. [5] "RouterArena: An Open Platform for Comprehensive Comparison of LLM Routers," arXiv:2510.00202, 2025. [6] Y. Li, "Rethinking Predictive Modeling for LLM Routing: When Simple kNN Beats Complex Learned Routers," arXiv:2505.12601, 2025. [7] H. Li et al., "LLMRouterBench: A Massive Benchmark and Unified Framework for LLM Routing," arXiv:2601.07206, 2026. [8] I. Ong et al., "RouteLLM: Learning to Route LLMs with Preference Data," ICLR 2025, arXiv:2406.18665. [9] "IRT-Router: Effective and Interpretable Multi-LLM Routing via Item Response Theory," ACL 2025, arXiv:2506.01048. [10] "Router-R1: Teaching LLMs Multi-Round Routing and Aggregation via Reinforcement Learning," NeurIPS 2025, arXiv:2506.09033. [11] W. Jitkrittum et al., "Universal Model Routing for Efficient LLM Inference" (UniRoute), arXiv:2502.08773, 2025. [12] "PILOT: Adaptive LLM Routing under Budget Constraints," EMNLP 2025 Findings, arXiv:2508.21141. [13] Y. Li, "LLM Bandit: Cost-Efficient LLM Generation via Preference-Conditioned Dynamic Routing," arXiv:2502.02743, 2025. [14] "BaRP: Learning to Route LLMs from Bandit Feedback," arXiv:2510.07429, 2025. [15] Bae, Son, Lee, "CQB-MNL / ACQB: Routing with User Retrials as Implicit Feedback," arXiv:2602.02061, 2026. [16] Lu et al., "The Routing Plateau: Understanding and Breaking the Accuracy Limits of LLM Routers," arXiv:2606.07587, 2026. [17] J. Lee, "Most of the Routing Gap Is Task Type" (14 models, 294 questions), arXiv:2608.23023, 2026. [18] "From Sampled Outcomes to Capability Distributions" (DARS), arXiv:2606.06924, 2026. [19] "RouteProfile," arXiv:2605.00180, 2026. [20] J. Dekoninck, M. Baader, M. Vechev, "A Unified Approach to Routing and Cascading for LLMs," ICML 2025, arXiv:2410.10347. [21] R. Zhuang et al., "EmbedLLM: Learning Compact Representations of Large Language Models," ICLR 2025, arXiv:2410.02223. [22] P. Zhou et al., "Agent-as-a-Router: Agentic Model Routing for Coding Tasks," arXiv:2606.22902, 2026. Code and CodeRouterBench: github.com/LanceZPF/agent-as-a-router. [23] "Arch-Router: Aligning LLM Routing with Human Preferences," arXiv:2506.16655, 2025. [24] F. Maia Polo et al., "tinyBenchmarks: Evaluating LLMs with Fewer Examples," arXiv:2402.14992, 2024. [25] "Avengers-Pro," arXiv:2508.12631, 2025. [26] "GraphRouter: A Graph-based Router for LLM Selections," ICLR 2025, arXiv:2410.03834 (*unverified*). [27] Braintrust, "The Best LLM Routers in 2026," <https://www.braintrust.dev/articles/best-llm-routers-2026>, accessed September 2026. [28] "RouterDC: Query-Based Router by Dual Contrastive Learning," NeurIPS 2024, arXiv:2409.19886 (*unverified*). [29] "MixLLM: Dynamic Routing in Mixed Large Language Models," NAACL 2025, arXiv:2502.18482 (*unverified*). [30] L. Chen, M. Zaharia, J. Zou, "FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance," arXiv:2305.05176 (*unverified*). [31] P. Aggarwal et al., "AutoMix:

Automatically Mixing Language Models," NeurIPS 2024, arXiv:2310.12963 (*unverified*). [32] L. Moslem, J. Kelleher, "Dynamic Model Routing and Cascading for Efficient LLM Inference: A Survey," TMLR 2026, arXiv:2603.04445.